

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

I Deepak V- 192411021 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear

1: XML Parsing Algorithm

Pseudocode:

```
import javax.xml.parsers.*;
```

```
import org.w3c.dom.*;
```

```
import java.io.*;
```

```
public class Main {
```

```
    public static void main(String[] args) throws Exception {
```

```
        String xml = ""
```

```
        <products>
```

```
            <product>
```

```
                <id>101</id>
```

```
                <name>Laptop</name>
```

```
                <price>55000</price>
```

```
            </product>
```

```
            <product>
```

```
                <id>102</id>
```

```
                <name>Phone</name>
```

```
                <price>30000</price>
```

```
            </product>
```

```
        </products>
```

```
        """,
```

```
        DocumentBuilderFactory f = DocumentBuilderFactory.newInstance();
```

```
        DocumentBuilder b = f.newDocumentBuilder();
```

```
        Document doc = b.parse(
```

```
            new ByteArrayInputStream(xml.getBytes())
```

```
        );
```

```
    NodeList products = doc.getElementsByTagName("product");

    System.out.println("Product Details");

    for (int i = 0; i < products.getLength(); i++) {

        Element p = (Element) products.item(i);

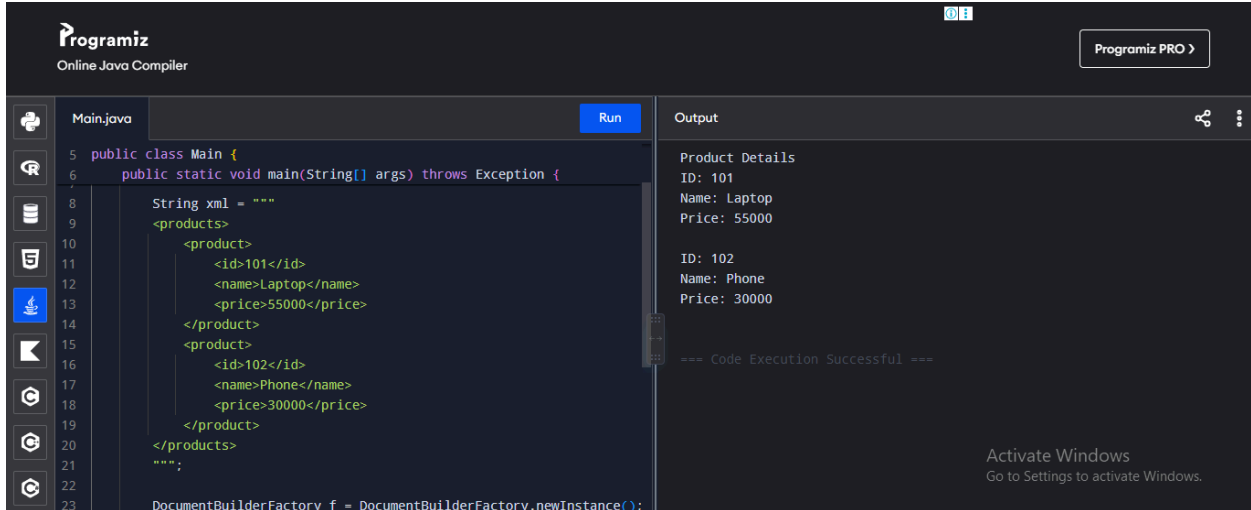
        String id = p.getElementsByTagName("id")
            .item(0).getTextContent();

        String name = p.getElementsByTagName("name")
            .item(0).getTextContent();

        String price = p.getElementsByTagName("price")
            .item(0).getTextContent();

        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Price: " + price);
        System.out.println();
    }
}
}
```

Output:



```
5 public class Main {
6     public static void main(String[] args) throws Exception {
7
8         String xml = ""
9         <products>
10             <product>
11                 <id>101</id>
12                 <name>Laptop</name>
13                 <price>55000</price>
14             </product>
15             <product>
16                 <id>102</id>
17                 <name>Phone</name>
18                 <price>30000</price>
19             </product>
20         </products>
21         """;
22
23         DocumentBuilderFactory f = DocumentBuilderFactory.newInstance();
```

Output

```
Product Details
ID: 101
Name: Laptop
Price: 55000

ID: 102
Name: Phone
Price: 30000

=== Code Execution Successful ===
```

2: MVC Workflow

Pseudocode:

```
public class Main {
```

```
// Model
```

```
static class StudentModel {
    String getStudentName() {
        return "Ram";
    }
}
```

```
// View
```

```
static class StudentView {
    void display(String name) {
        System.out.println("Student Details");
        System.out.println("Name: " + name);
    }
}
```

```
// Controller
static class StudentController {
    StudentModel model;
    StudentView view;

    StudentController(StudentModel model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    void processRequest() {
        System.out.println("Request received by Controller");

        String name = model.getStudentName();

        System.out.println("Model processed the request");

        view.display(name);

        System.out.println("Response sent to User");
    }
}

public static void main(String[] args) {

    StudentModel model = new StudentModel();
```

```

        StudentView view = new StudentView();

        StudentController controller =
            new StudentController(model, view);

        controller.processRequest();
    }
}

```

Output:

The screenshot shows the Programiz Online Java Compiler interface. The code editor on the left contains the following Java code:

```

1 public class Main {
4     static class StudentModel {
7     }
8
9     // View
10
11     static class StudentView {
12         void display(String name) {
13             System.out.println("Student Details");
14             System.out.println("Name: " + name);
15         }
16     }
17
18     // Controller
19     static class StudentController {
20         StudentModel model;
21         StudentView view;
22
23         StudentController(StudentModel model, StudentView view) {

```

The output window on the right shows the following text:

```

Request received by Controller
Model processed the request
Student Details
Name: Madara Uchiha
Response sent to User

=== Code Execution Successful ===

```

At the bottom right, there is a message: "Activate Windows Go to Settings to activate Windows."

3: JSP Request Flow

Pseudocode:

```

public class Main {

    static class Database {

        static String getData() {

            return "Welcome to the Student Portal";

        }

    }

}

```

```
}
```

```
static class Servlet {
```

```
    static String processRequest() {
```

```
        System.out.println("Servlet processing request...");
```

```
        return Database.getData();
```

```
    }
```

```
}
```

```
static class JSP {
```

```
    static void generateResponse(String data) {
```

```
        System.out.println("JSP generating dynamic content...");
```

```
        System.out.println("Response: " + data);
```

```
    }
```

```
}
```

```
public static void main(String[] args) {
```

```
    System.out.println("Client sends JSP request");
```

```
    String data = Servlet.processRequest();
```

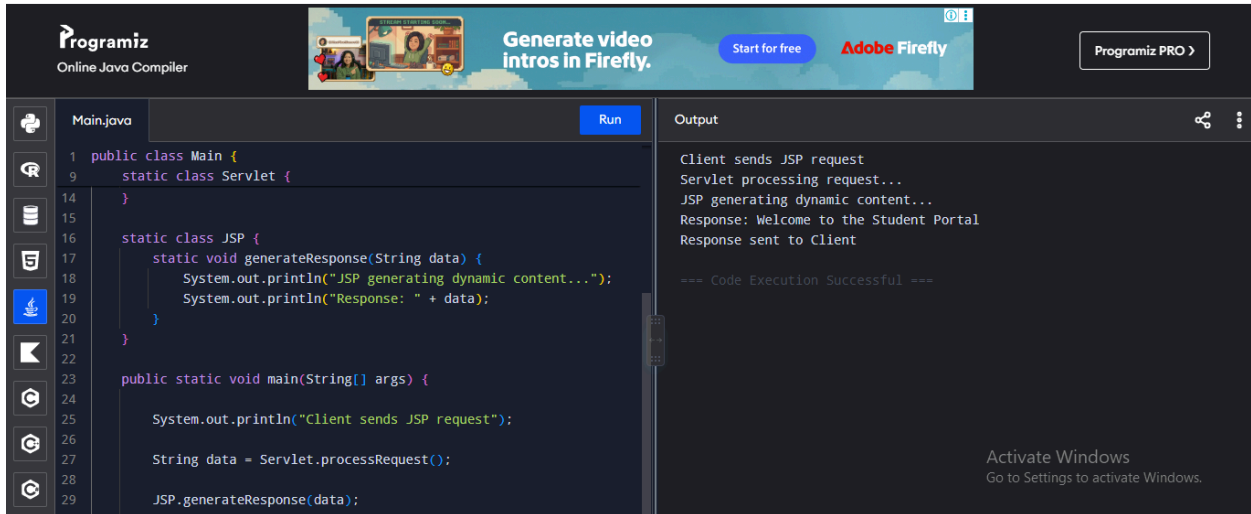
```
    JSP.generateResponse(data);
```

```
    System.out.println("Response sent to Client");
```

```
}
```

```
}
```


Output:



The screenshot shows the Programiz Online Java Compiler interface. At the top, there's a banner for "Generate video intros in Firefly" with a "Start for free" button and the Adobe Firefly logo. Below the banner, the editor shows a file named "Main.java" with the following code:

```
1 public class Main {
9     static class Servlet {
14     }
15
16     static class JSP {
17         static void generateResponse(String data) {
18             System.out.println("JSP generating dynamic content...");
19             System.out.println("Response: " + data);
20         }
21     }
22
23     public static void main(String[] args) {
24
25         System.out.println("Client sends JSP request");
26
27         String data = Servlet.processRequest();
28
29         JSP.generateResponse(data);
30     }
31 }
```

The output window on the right shows the following text:

```
Client sends JSP request
Servlet processing request...
JSP generating dynamic content...
Response: Welcome to the Student Portal
Response sent to Client

=== Code Execution Successful ===
```

At the bottom right, there's a message: "Activate Windows. Go to Settings to activate Windows."

4: Database Retrieval

Pseudocode:

```
public class Main {
```

```
    static String[][] database = {
        {"101", "Laptop", "55000"},
        {"102", "Phone", "30000"},
        {"103", "Tablet", "25000"}
    };
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Database Connected");
        System.out.println("Executing SELECT Query...\n");
```

```
        System.out.println("ID\tProduct\tPrice");
```

```

for(String[] row : database) {
    System.out.println(
        row[0] + "\t" +
        row[1] + "\t" +
        row[2]
    );
}

System.out.println("\nData Retrieved Successfully");
System.out.println("Database Connection Closed");
}
}

```

Output:

The screenshot displays the Programiz Online Java Compiler interface. The code editor on the left contains the following Java code:

```

1 public class Main {
2
3     static String[][] database = {
4         {"101", "Laptop", "55000"},
5         {"102", "Phone", "30000"},
6         {"103", "Tablet", "25000"}
7     };
8
9     public static void main(String[] args) {
10
11         System.out.println("Database Connected");
12         System.out.println("Executing SELECT Query...\n");
13
14         System.out.println("ID\tProduct\tPrice");
15
16         for(String[] row : database) {
17             System.out.println(
18                 row[0] + "\t" +
19                 row[1] + "\t" +
20                 row[2]
21             );
22         }
23
24         System.out.println("\nData Retrieved Successfully");
25         System.out.println("Database Connection Closed");
26     }
27 }

```

The output window on the right shows the following results:

```

Database Connected
Executing SELECT Query...

ID Product Price
101 Laptop 55000
102 Phone 30000
103 Tablet 25000

Data Retrieved Successfully
Database Connection Closed

=== Code Execution Successful ===

```

At the top of the interface, there are banners for Programiz, a promotion for smooth sailing through 20+ apps, and an Adobe Creative Cloud Pro advertisement. A 'Programiz PRO' button is also visible.

5: PHP Validation

Pseudocode:

```
<?php
```

```
$name = "Ram";
```

```
$email = "Ram@gmail.com";
```

```
$password = "Pass1234";
```

```
$errors = [];
```

```
if (empty($name))
```

```
    $errors[] = "Name is required";
```

```
if (!filter_var($email, FILTER_VALIDATE_EMAIL))
```

```
    $errors[] = "Invalid email";
```

```
if (strlen($password) < 6)
```

```
    $errors[] = "Password must contain at least 6 characters";
```

```
if (empty($errors)) {
```

```
    echo "Validation Successful<br>";
```

```
    echo "Form submitted successfully";
```

```
} else {
```

```
    echo "Validation Failed<br>";
```

```
    foreach ($errors as $error)
```

```
        echo $error . "<br>";
```

```
}  
?>
```

Output:



The screenshot displays the Programiz PHP Online Compiler interface. At the top, there are banners for Programiz, Adobe Creative Cloud Pro, and a student discount offer. The main area is divided into two panels: a code editor on the left and an output panel on the right. The code editor shows a PHP script named 'main.php' with the following code:

```
6  
7 $errors = [];  
8  
9 if (empty($name))  
10     $errors[] = "Name is required";  
11  
12 if (!filter_var($email, FILTER_VALIDATE_EMAIL))  
13     $errors[] = "Invalid email";  
14  
15 if (strlen($password) < 6)  
16     $errors[] = "Password must contain at least 6 characters";  
17  
18 if (empty($errors)) {  
19     echo "Validation Successful<br>";  
20     echo "Form submitted successfully";  
21 } else {  
22     echo "Validation Failed<br>";  
23 }
```

The output panel shows the result of the script execution:

```
Validation Successful<br>Form submitted successfully  
=== Code Execution Successful ===
```

At the bottom right of the output panel, there is a message: "Activate Windows Go to Settings to activate Windows."